



Natural Language Processing

Text Classification & POS Tagging with MLPs

Nov 2025

**Maria Schoinaki
Electra Papadopoulou
Leonidas Kontogiannis**

You can view the full code implementation for the **text classification** in the following link:



You can view the full code implementation for the **POS tagger** in the following link:



All members have contributed equally in all tasks, markdown and report writing, synchronously throughout the assignment.

Tweet Sentiment Classification with MLPs

For this assignment we used the Twitter Sentiment Analysis dataset from Kaggle. This dataset consists of ~100.000 tweets (sentiment texts), the id of each tweet, and the sentiment (0 or 1), and it was used as the basis for training (70%), development (15%), and testing (15%) of the sentiment classifiers. The same dataset had been used for **Assignment 2, “Tweet Sentiment Analysis”**, thus we will be brief in summarizing the preprocess and baseline classifier stage. For more details, you can read the report [here](#).

Data Exploration

As shown in **Assignment 2**, the dataset is roughly balanced ($\approx 43.5\%$ negative, $\approx 56.5\%$ positive), and the tweets are short (mean ≈ 13 words), which is typical for Twitter.

Data Preprocessing/ Cleaning

In order to preprocess the dataset, we implement **spaCy** and **regular expressions**.

- **Text Cleaning:**

For a dataset full of tweets, it is important to scrap the information that is not useful for sentiment classification, such as **symbol-prefixed words**; we eliminate mentions, tagging, quoting, HTML entities and URLs.. Non-word characters and extra spaces are also removed and lowercasing is applied to ensure there's case consistency across tokens.

- **Linguistic Normalization:**

To preprocess each tweet, we will utilize **Part-Of-Speech (POS)** tagging, specifically spaCy's **English language model** (*en_core_web_sm*). Only **tokenization** and **lemmatization** were required.

The result of the process is a list of tweets. Here are some examples of the results:

Original tweet text:

@akiraLOVE haha, yeah hey, he replied to you too right??

Filtered tweet text:

haha yeah hey he reply to you too right

Original tweet text:

@ashers11 Whyy?

Filtered tweet text:

why

Experimental Setup

For Exercise 4 we reuse exactly the same dataset, splits, and preprocessing pipeline as in Exercise 11 (Part 2):

- **Corpus:** twitter-sentiment-analysis2 (Kaggle), with ~100K tweets labeled with binary sentiment (0 = Negative, 1 = Positive).
- **Splits:** 70% train, 15% development, 15% test, stratified by label, using random_state=2025.
- **Preprocessing:** Text Cleaning and Linguistic Normalization, as described above,

For **Exercise 4 (Assignment 3)**, our goal is to replace the purely linear classifiers with a non-linear MLP implemented in PyTorch, and compare it to:

- A majority baseline (always predicts the most frequent label in the training set).
- Our best linear model from **Assignment 2**: a TF-IDF + Logistic Regression classifier.

Feature Extraction for the MLP

TF-IDF n-gram features

We first convert the cleaned tweets into TF-IDF n-gram vectors using scikit-learn's `TfidfVectorizer`:

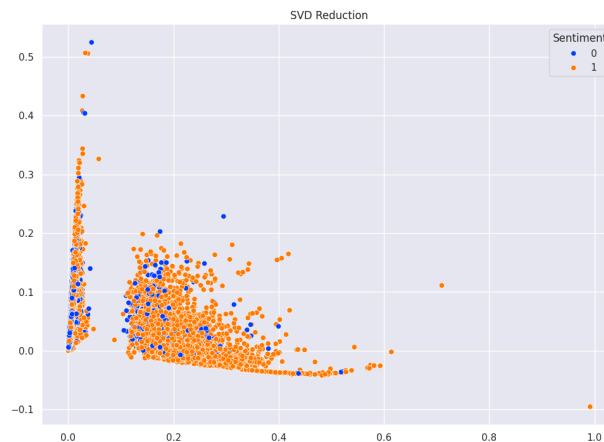
```
vectorizer = FeatureUnion([
    ('word', TfidfVectorizer(ngram_range = (1, 3), max_features = 30000, stop_words='english', sublinear_tf = True)),
    # Character features help with misspellings, slang, and emojis.
    ('char', TfidfVectorizer(analyzer='char', ngram_range=(3,5), max_features=20000))
])
```

This yields high-dimensional sparse vectors: about **30,000 features per tweet**.

Dimensionality reduction for the MLP

Directly feeding 30K-dimensional sparse vectors into a PyTorch MLP would be inefficient. Instead, we project the TF-IDF vectors into a lower-dimensional dense space using Truncated SVD (`TruncatedSVD`):

- `n_components = 500`
- `random_state = 2025`



We fit on the training TF-IDF matrix and transform all splits. The MLP therefore operates on **500-dimensional dense vectors**. This is analogous to using latent semantic features and also helps regularize the model.

Baseline Classifiers

We consider two baselines, as requested:

Majority baseline

- Implemented with scikit-learn's `DummyClassifier(strategy="most_frequent")`.
- Trained on raw preprocessed text (labels only).

- Serves as a lower bound for performance.

TF-IDF + Logistic Regression (best model from Exercise 11)

This is the same model we tuned in the previous assignment:

- Feature selection: `SelectKBest(mutual_info_classif, k=30,000)`
- Classifier: `LogisticRegression` with `C=1` (best value found in Exercise 11)

We reuse this tuned **TF-IDF + LR** as our strong linear baseline and retrain it on the same train/dev/test split used for the MLP.

MLP Architecture and Training

Network architecture

The final best configuration (after tuning) is:

- Input layer: 500-dimensional SVD features.
- Hidden layers: a single fully connected layer with:
 - `hidden_sizes = [512]`
 - Activation: `ReLU`
 - Dropout: tuned value (see below; best ≈ 0.7 in our experiments).
- Output layer: linear layer with 2 units (Negative, Positive), trained with cross-entropy loss.

Overall, the MLP is a relatively small network but still non-linear, able to model interactions between latent SVD features.

Data loaders

We convert the SVD features and labels to PyTorch tensors:

- `X_train_svd, X_dev_svd, X_test_svd` $\rightarrow$ `FloatTensor`
- `labels` $\rightarrow$ `LongTensor`

We build `TensorDatasets` and `DataLoaders`:

- Batch size: 256
- Train loader shuffled, dev/test loaders not shuffled.

This allows standard mini-batch training with stochastic gradient descent.

Hyperparameter Tuning

We tune three main aspects on the **development set**:

1. Number and size of hidden layers

We evaluate architectures: [512], [512, 256], [1024, 512, 256], [256], [1024, 1024]

For each architecture we train up to 250 epochs with:

- Optimizer: Adam with learning rate $lr = 1e-3$.
- Loss: CrossEntropyLoss.
- Early stopping based on dev loss (patience = 5 epochs).

For each configuration we log: **Training Loss**, **Development Loss** and **Development Accuracy**.

The simple architecture with a single hidden layer [512] achieves **competitive dev accuracy** with fewer parameters, and we choose it as a good trade-off between capacity and regularization.

2. Dropout Probability

Fixing the hidden layer at [512], we tune dropout with a dedicated function **tune_dropout**, evaluating a range of values between **0.1** and **1.0**. For each dropout value we:

- Train the MLP on the training set.
- Monitor train and dev loss.
- Track dev accuracy and apply early stopping.

We then select the dropout value with the **highest dev accuracy**. In our experiments, a **high dropout** (around **0.7**) works best, indicating that regularization is important on this relatively small dataset and low-dimensional SVD features.

3. Number of Epochs

After choosing architecture [512] and the best dropout, we perform **epoch tuning** with the function `epoch_tuning`:

- Train for up to 200 epochs,
- Record train loss, dev loss, and dev accuracy at each epoch.

- Select the epoch with **maximum dev accuracy**.

This best epoch is then used in the final training run.

4. Final MLP Training

We train the **final MLP** with:

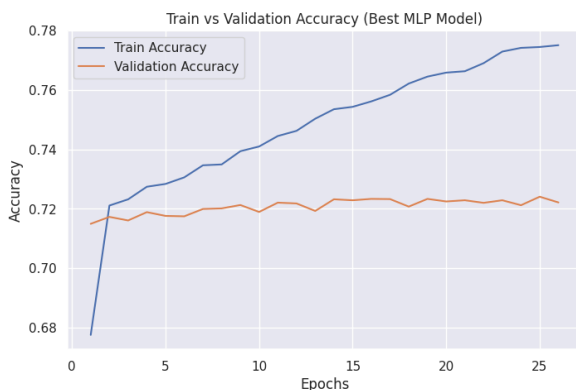
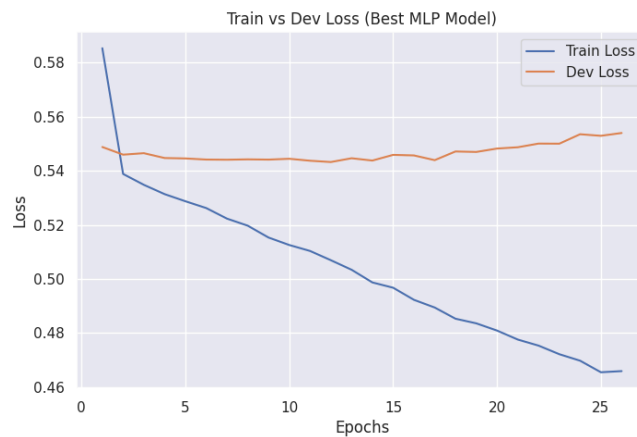
- Hidden sizes: [512]
- Dropout: best value from dropout tuning (≈ 0.7)
- Epochs: best value from epoch tuning (≈ 26 in our run)
- Optimizer: Adam(lr=1e-3)

During this final training we log: **Training loss, Dev loss, Training accuracy & F1 (macro)** and **Dev accuracy & F1 (macro)**. We then keep the final weights (we could also keep best dev epoch weights; in practice these coincide or are very close).

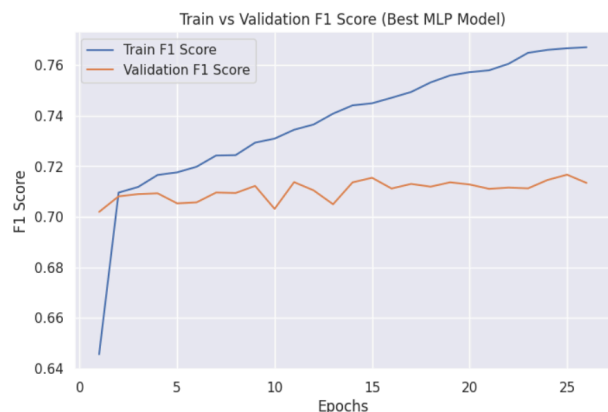
Loss Curves

After training, we analyze the behavior of the best model across epochs by visualizing the training history:

- **Train vs. Dev Loss:** The curves help determine whether the model continues learning (**training loss decreasing**), whether it generalizes well (**dev loss follows train loss without diverging**) and whether overfitting occurs (**dev loss increases while train loss continues to drop**).



- **Train vs. Validation Accuracy:** We want to check if the model is consistently improving its predictions.



- **Train vs. Validation F1 Score:** It monitors whether the model improves in its ability to handle all classes consistently, and whether precision–recall tradeoffs change over training, though it is more informative when classes have uneven representation.

Additionally, all of these curves can show when the model starts overfitting, which is happening in this case, even though we have tuned our model to the best epoch. The dev loss does not follow the train loss, and thus the validation accuracy and F1-score do not improve that greatly over time.

Evaluation

We report per-class **Precision (P)**, **Recall (R)**, **F1**, and **PR-AUC** and we also include **Macro-Averaged P/R/F1/PR-AUC**. Models are **trained** on *train* dataset, **tuned** in *dev* dataset and **evaluated** in *test* dataset.

Systems Compared

- **Majority baseline** (*dummy classifier-most_frequent*): Always predicts the training set’s majority class.
- **LR (TF-IDF) tuned:** Logistic Regression with TF-IDF word n-grams (*uni, bi, tri*).
- **MLP (TF-IDF) tuned:** Multi-Layered Perceptron with SVD features.

Results

We provide three tables (*train, dev, test*). Each table has one row per model and four columns per class (*P, R, F1, PR-AUC*), followed by *Macro-P, Macro-R, Macro-F1, Macro-PR-AUC*.

Training Set

=== TRAIN RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435364	0.564636	1.000000	0.721747	0.564636	0.282318	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.793206	0.827645	0.810060	0.878431	0.862502	0.833629	0.847820	0.930190	0.827854	0.830637	0.828940	0.904310
2	MLP	0.818141	0.725486	0.769033	0.868709	0.805334	0.875658	0.839025	0.911225	0.811738	0.800572	0.804029	0.889967

- The **majority baseline** reaches Macro-F1 ≈ 0.36 and Macro PR-AUC ≈ 0.50 (*constant prediction yields poor F1 but PR-AUC near the class prior*).

- The LR model achieves a high training score that surpasses our MLP, which could be worrisome, but it is also expected, as our model was prone to overfitting, especially on low-dimensional SVD features derived from sparse TF-IDF vectors.

Development Set

=== DEV RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435391	0.564609	1.000000	0.721725	0.564609	0.282304	0.500000	0.360863	0.500000
1	TF-IDF+LR	0.722174	0.757121	0.739234	0.805588	0.805447	0.775390	0.790132	0.876435	0.763810	0.766255	0.764683	0.841011
2	MLP	0.702069	0.628637	0.663327	0.746026	0.735002	0.794284	0.763494	0.827080	0.718536	0.711461	0.713411	0.786553

- **Baseline:** Macro F1≈0.3609, Macro PR-AUC=0.5000
- **LR (TF-IDF):** P=0.7638, R=0.7666, F1=0.7646, PR-AUC=0.8410
- **MLP (TF-IDF):** Macro F1=0.7134, Macro PR-AUC=0.7865

Observation: TF-IDF LR clearly dominates the MLP on Dev (≈+5.1 Macro-F1, +0.081 PR-AUC). The baseline is far behind, as expected.

Test Set

=== TEST RESULTS ===													
	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435362	0.564638	1.000000	0.721749	0.564638	0.282319	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.726814	0.750077	0.738262	0.810192	0.802421	0.782619	0.792396	0.875979	0.764618	0.766348	0.765329	0.843085
2	MLP	0.703993	0.620980	0.659886	0.753566	0.732114	0.798678	0.763948	0.825389	0.718053	0.709829	0.711917	0.789477

The **baseline** predicts only the majority class → **zero** performance for the minority class and **flat PR-AUC ~ class prior**.

The MLP achieves slightly lower training performance than LR and suffers a somewhat larger generalization gap (train vs dev/test), suggesting mild overfitting and that it is less effective than LR in this sparse TF-IDF setting. (LR dominates under TF-IDF because sparse, high-dimensional representations favor linear decision boundaries. However, with dense pretrained embeddings, the MLP can learn meaningful nonlinear interactions and is likely to outperform LR. Therefore, using pretrained embeddings would be a promising direction for future improvement.

Best overall: LR (TF-IDF) with Macro-F1=0.765 and Macro PR-AUC=0.843.

POS Tagging

For this assignment we used the UD Ancient Greek PROIEL treebank from Universal Dependencies. The corpus contains morphologically and syntactically annotated **Ancient Greek** texts. After loading the dataset, we obtained **15,016** sentences for training, **1,019** sentences for development, and **1,047** sentences for testing. Each sentence consists of a sequence of tokens paired with their corresponding Universal POS tags (UPOS).

Data Preprocessing

We **tokenize** and **pad** sentences to fixed-sized windows of ± 2 words around the target word, applying special padding tokens at sentence boundaries. Using the training set, we then built two vocabularies: one for words and one for the UPOS tag set. The word vocabulary assigned a unique index to every token appearing in the training data, while also reserving special symbols for padding and unknown words. The tag vocabulary included all UPOS labels observed in the training set and mapped each label to a numerical index. These vocabularies were later used to convert sentences into sequences of indices suitable as input to the neural model. We used 2 methods for creating our word embeddings: pre-trained word embeddings from word2vec and random embeddings trained by our MLP. We chose the pre-trained word2vec.

The following table summarizes **key statistics** for the training, development, and test subsets of the **Ancient Greek PROIEL** treebank:

```
TRAIN stats:
sentences:      15016
tokens:         187039
avg sent length: 12.46
median sent len: 10.00
vocab size:     30349
number of tags: 13
most common tags:
  VERB: 34517
  NOUN: 31424
  DET: 28070
  ADV: 20288
  PRON: 19756
  ADP: 13812
  CCONJ: 12117
  ADJ: 10361
  PROPN: 7745
  SCONJ: 3610
```

```
DEV stats:
sentences:      1019
tokens:         13652
avg sent length: 13.40
median sent len: 11.00
vocab size:     4541
number of tags: 13
most common tags:
  VERB: 2671
  NOUN: 2185
  DET: 2082
  PRON: 1446
  ADV: 1378
  CCONJ: 978
  ADP: 922
  ADJ: 729
  PROPN: 618
  SCONJ: 257
```

```
TEST stats:
sentences:      1047
tokens:         13314
avg sent length: 12.72
median sent len: 11.00
vocab size:     4495
number of tags: 13
most common tags:
  VERB: 2494
  NOUN: 2209
  DET: 2006
  PRON: 1427
  ADV: 1416
  CCONJ: 1011
  ADP: 885
  ADJ: 752
  PROPN: 494
  AUX: 248
```

Word Embeddings:

We trained **Word2Vec** embeddings directly on the **training corpus**. Only the word sequences were used as input for Word2Vec, and we employed a **skip-gram architecture** with a vector dimension of **100**, a window size of 5, and a minimum frequency threshold of one. The Word2Vec model was trained for ten epochs. This produced distributional word vectors for all words seen in the training data.

Our Model

The POS tagger is a window-based multilayer perceptron (MLP) that receives the ± 2 -word context window as input. Each word index is first mapped to a continuous vector through an embedding layer, initialized with the pretrained Word2Vec matrix and fine-tuned during training. The window embeddings are concatenated and passed through a stack of fully connected layers with ReLU activations and dropout regularization. The final linear layer outputs class logits for all UPOS tags. This architecture allows the model to combine local contextual information with pretrained lexical features to predict the tag of the target word.

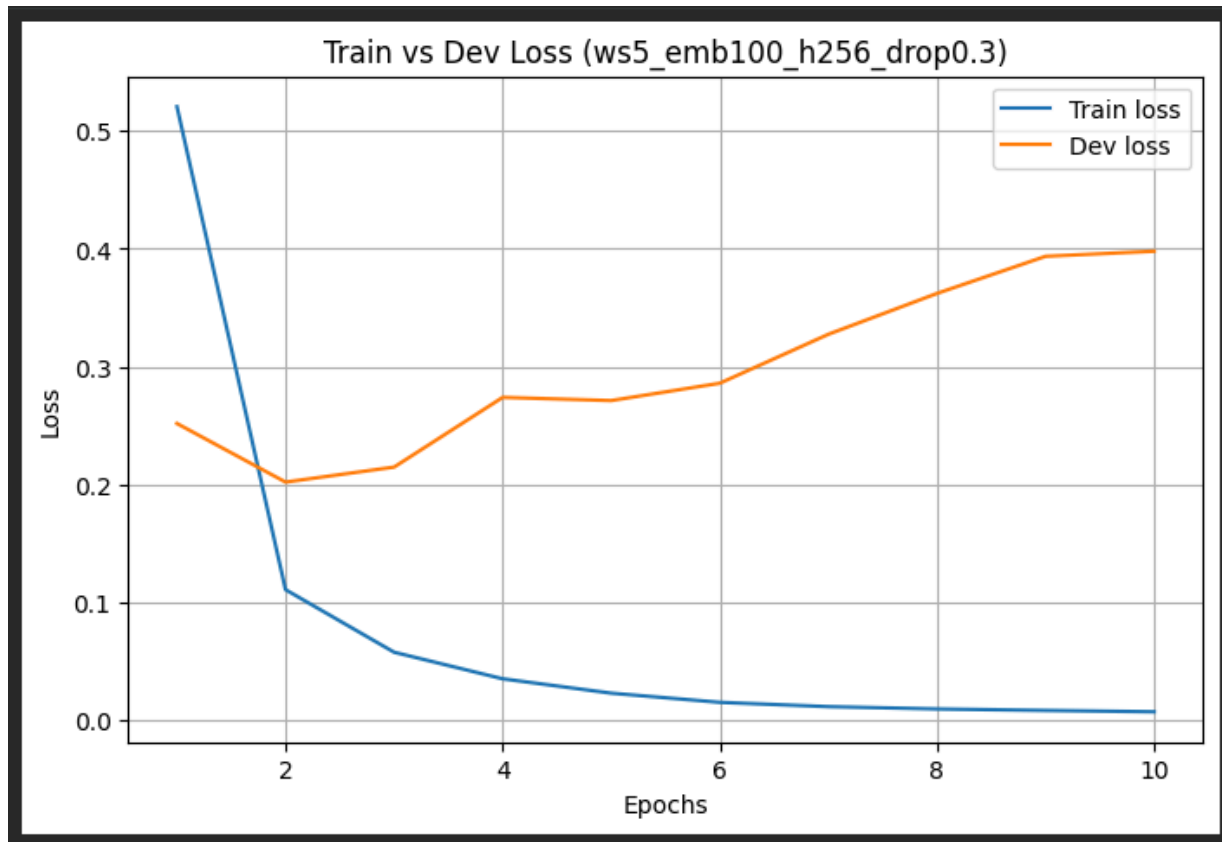
Training was performed in mini-batches using the Adam optimizer and cross-entropy loss. After each epoch, performance was evaluated on a development set using both loss and macro-F1 score.

Early stopping was applied: if the development loss increased consistently over a predefined number of consecutive epochs (patience), training was halted. The model weights corresponding to the best development macro-F1 score were saved and restored at the end of training. This approach ensured that the final model achieved strong generalization performance while avoiding unnecessary overfitting.

Fine Tuning

To identify the optimal model configuration, we performed a hyperparameter search over several network settings, hidden layer sizes and dropout. For each configuration, the model was trained on the training set and evaluated on the development set using macro-F1.

All configurations were trained separately, and their training histories (loss curves) were recorded. The model achieving the highest macro-F1 on the development set was selected as the best model. This systematic approach allowed us to fine-tune the architecture and training parameters to maximize generalization performance. Our best model uses a 5-word window, 100-d embeddings, one hidden layer with 256 units and dropout 0.3.



The training loss decreases monotonically, while the development loss initially decreases and then rises again, indicating overfitting if we were to train for too many epochs. However, by selecting the epoch with the best development macro-F1, the final model corresponds to an early epoch before severe overfitting sets in.

=== TRAIN RESULTS ===																					
	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9845	0.9908	0.9876	0.9759	0.9765	0.9940	0.9852	0.9711	0.9726	...	0.8797	0.7775	0.9996	0.9992	0.9994	0.9989	0.9671	0.9665	0.9664	0.9384
1	LogReg (window)	0.9905	0.9944	0.9924	0.9992	0.9895	0.9968	0.9931	0.9996	0.9947	...	0.9654	0.9919	0.9983	0.9998	0.9990	1.0000	0.9892	0.9863	0.9877	0.9985
2	MLP (best config)	0.9985	0.9902	0.9943	0.9999	0.9969	0.9987	0.9978	1.0000	0.9972	...	0.9819	0.9981	0.9999	0.9999	0.9999	1.0000	0.9933	0.9941	0.9937	0.9996
3 rows x 57 columns																					
=== DEV RESULTS ===																					
	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9774	0.7133	0.8247	0.7125	0.9683	0.9935	0.9807	0.9624	0.9556	...	0.8419	0.7201	0.7957	0.9989	0.8858	0.7950	0.9489	0.9039	0.9211	0.8627
1	LogReg (window)	0.8987	0.7421	0.8129	0.9030	0.9818	0.9924	0.9871	0.9991	0.9748	...	0.9160	0.9659	0.8314	0.9948	0.9057	0.9859	0.9475	0.9223	0.9329	0.9757
2	MLP (best config)	0.9430	0.7257	0.8202	0.9029	0.9892	0.9946	0.9919	0.9991	0.9743	...	0.9290	0.9730	0.8551	0.9885	0.9174	0.9857	0.9504	0.9293	0.9379	0.9756
3 rows x 57 columns																					
=== TEST RESULTS ===																					
	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9730	0.7660	0.8571	0.7585	0.9723	0.9932	0.9827	0.9662	0.9746	...	0.8904	0.7953	0.7987	0.9976	0.8871	0.7972	0.9523	0.9197	0.9325	0.8811
1	LogReg (window)	0.9096	0.8032	0.8531	0.9224	0.9844	0.9966	0.9905	0.9992	0.9782	...	0.9310	0.9770	0.8381	0.9944	0.9096	0.9833	0.9467	0.9278	0.9357	0.9772
2	MLP (best config)	0.9438	0.7819	0.8553	0.9229	0.9866	0.9955	0.9910	0.9998	0.9768	...	0.9248	0.9781	0.8576	0.9832	0.9161	0.9809	0.9486	0.9367	0.9414	0.9774
3 rows x 57 columns																					

- On **train**, MLP > LogReg > baseline
- On **dev/test**, the MLP seems to slightly outperform the window LogReg in macro-F1 and PR-AUC, but all three are very strong overall.
- The per-word baseline is strong on frequent tags but weaker on rarer ones.
- The window-based logistic regression boosts performance by using local context plus embeddings.
- The MLP adds non-linear interactions and achieves the best macro-F1 / macro-PR-AUC, at the cost of being more prone to overfitting (as seen in the loss curves).

The MLP and window-based logistic regression significantly outperform the baseline in macro-F1 on all splits, especially for rarer tags.